



---

---

---

---

---

---

---

---



---

---

---

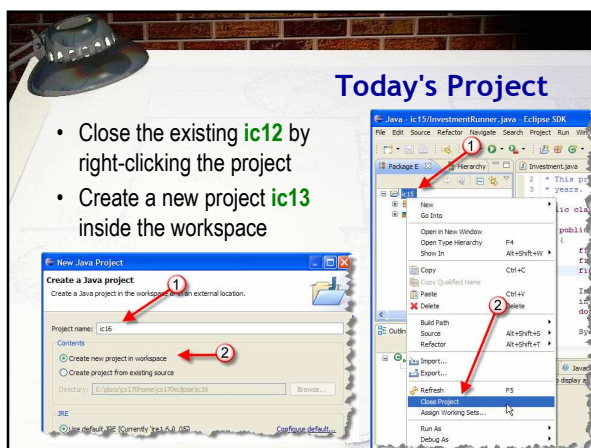
---

---

---

---

---



---

---

---

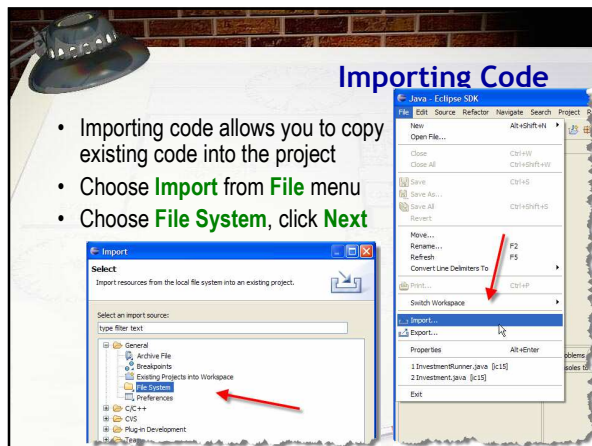
---

---

---

---

---




---

---

---

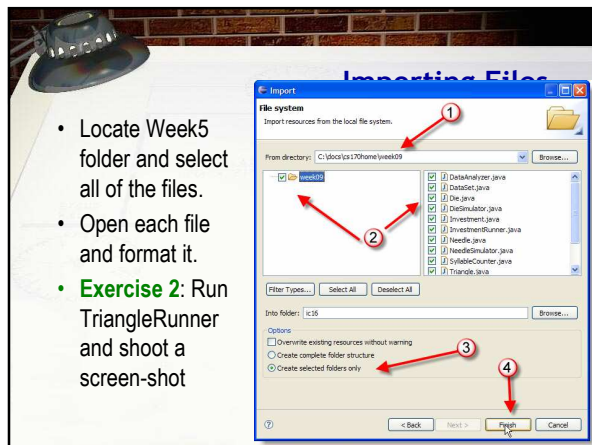
---

---

---

---

---




---

---

---

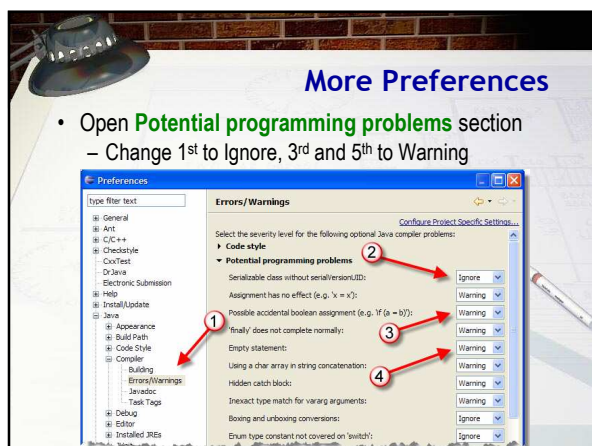
---

---

---

---

---




---

---

---

---

---

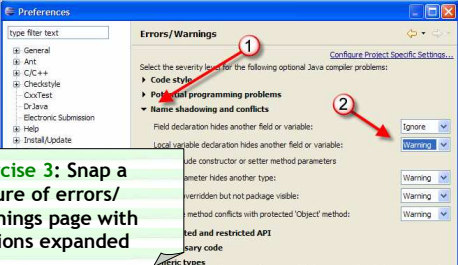
---

---

---

## Name Conflicts

- Choose **Name shadowing and conflicts** section
  - Change 2<sup>nd</sup> option to Warning, read through others



**Exercise 3:** Snap a picture of errors/Warnings page with sections expanded

---

---

---

---

---

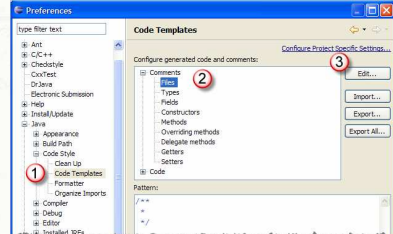
---

---

---

## Code Templates

- You can also configure "custom comments"
  - Apply to types, classes, specific kinds of methods, etc.
- Let's modify the "file", "type" and "method" comments




---

---

---

---

---

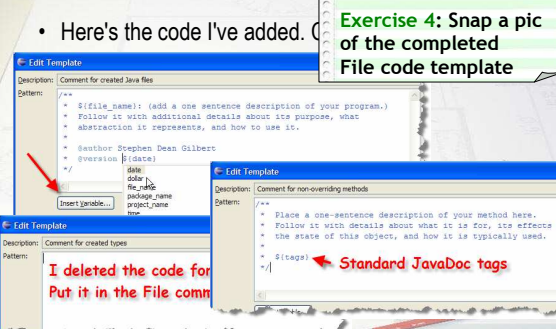
---

---

---

## The Comments

- Here's the code I've added.



**Exercise 4:** Snap a pic of the completed File code template

I deleted the code for... Put it in the File comm...

Standard JavaDoc tags

---

---

---

---

---

---

---

---

### Writing New Code

- Real power of Eclipse comes with code from scratch
  - We'll see how to create several kinds of programs
- Start with a simple console program that uses a counted loop to produce this output when run:
 

```
Enter a positive integer: 3
1 I love Java programming!!
2 I love Java programming!!
3 I love Java programming!!
Printed this message 3 times.
The sum of the numbers from 1 to 3 is 6.
```

---

---

---

---

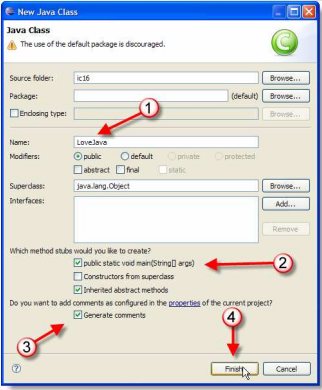
---

---

---

---

- Click the **New Java Class** toolbar button
- Name: LoveJava
- Main method checked
- Generate comments checked
- Finish




---

---

---

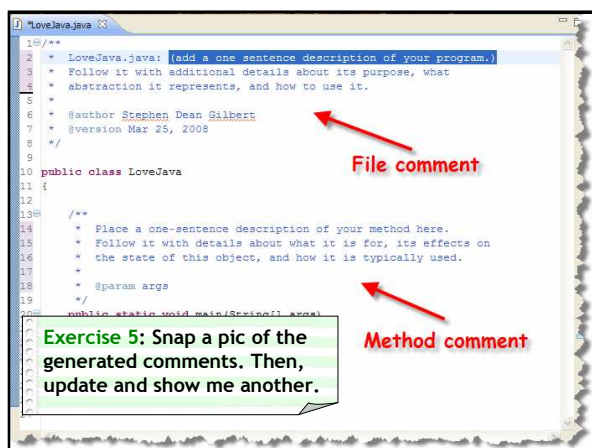
---

---

---

---

---



**File comment**

**Method comment**

**Exercise 5:** Snap a pic of the generated comments. Then, update and show me another.

---

---

---

---

---

---

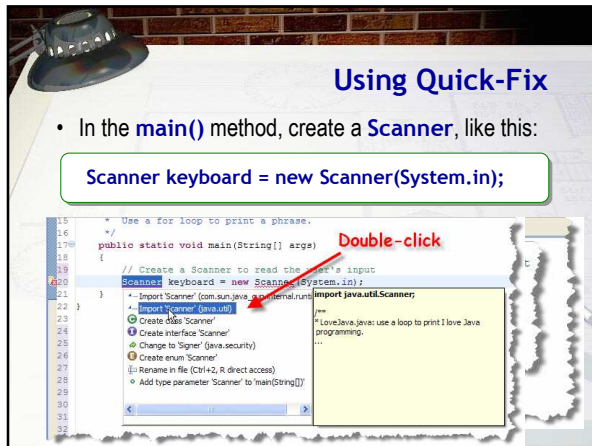
---

---

### Using Quick-Fix

- In the **main()** method, create a **Scanner**, like this:

```
Scanner keyboard = new Scanner(System.in);
```



Double-click

---

---

---

---

---

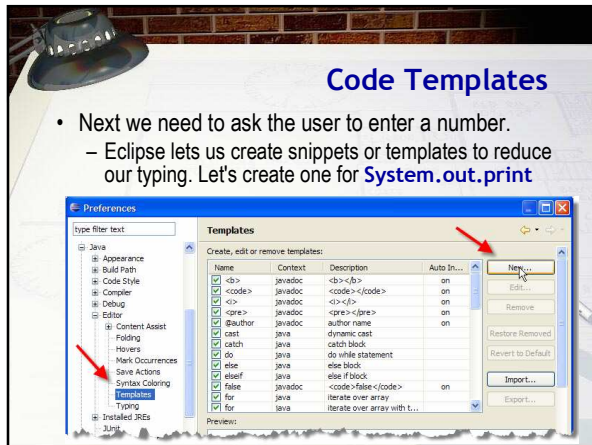
---

---

---

### Code Templates

- Next we need to ask the user to enter a number.
  - Eclipse lets us create snippets or templates to reduce our typing. Let's create one for **System.out.print**



---

---

---

---

---

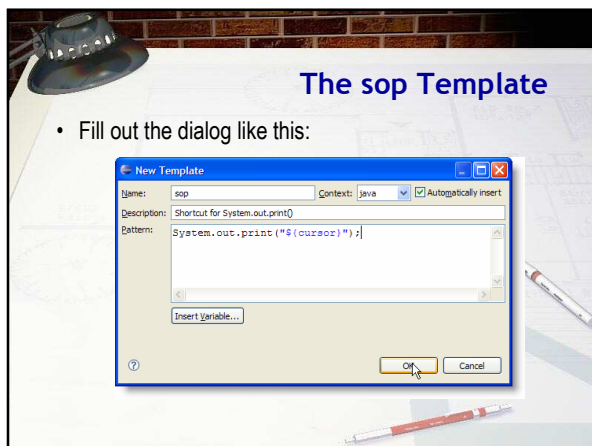
---

---

---

### The sop Template

- Fill out the dialog like this:



---

---

---

---

---

---

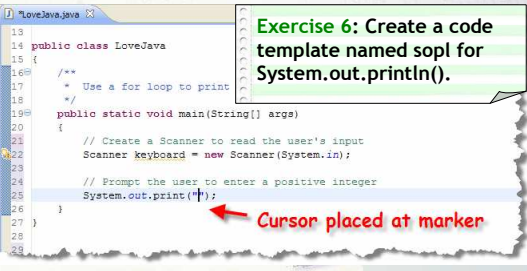
---

---



### Using sop

- Type **sop** and then **Control + Space** together
  - Inserts template snippet into your code



**Exercise 6:** Create a code template named **sopl** for **System.out.println()**.

Cursor placed at marker

---

---

---

---

---

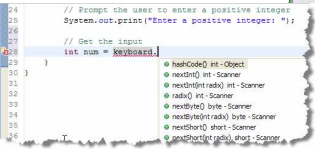
---

---

---

### Method Help

- Use the **Scanner** to find out how many times to loop
 

```
int num = keyboard.
```
- Typing a dot after an **object name** brings up a list of **instance methods** that you can use
 
- Use arrows, mouse, continue typing, **Enter** to select

---

---

---

---

---

---

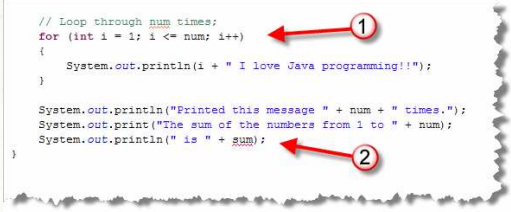
---

---

### Finishing Up

**Exercise 7:** Fix the error below by creating a variable **sum**, and adding up the numbers in the loop. Snap a pic.

- Now, add a **for** loop that goes from **1** to **n**
  - Print your message inside the loop body
  - After the loop print the two "summary" lines




---

---

---

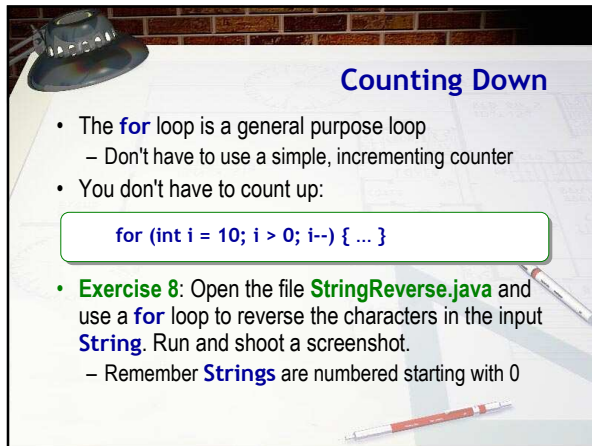
---

---

---

---

---



### Counting Down

- The **for** loop is a general purpose loop
  - Don't have to use a simple, incrementing counter
- You don't have to count up:

```
for (int i = 10; i > 0; i--) { ... }
```
- Exercise 8:** Open the file **StringReverse.java** and use a **for** loop to reverse the characters in the input **String**. Run and shoot a screenshot.
  - Remember **Strings** are numbered starting with 0

---

---

---

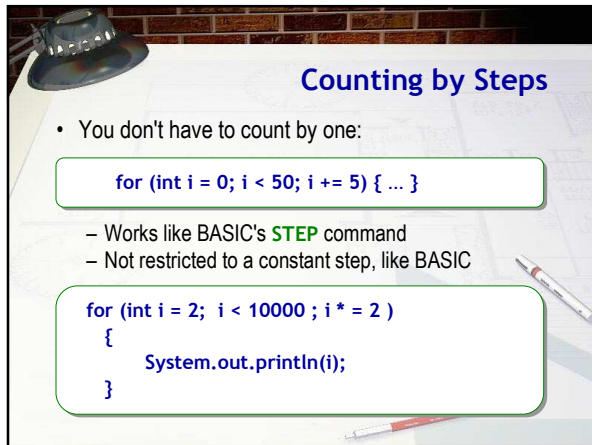
---

---

---

---

---



### Counting by Steps

- You don't have to count by one:

```
for (int i = 0; i < 50; i += 5) { ... }
```

  - Works like BASIC's **STEP** command
  - Not restricted to a constant step, like BASIC
- ```
for (int i = 2; i < 10000 ; i * = 2 )  
{  
    System.out.println(i);  
}
```

---

---

---

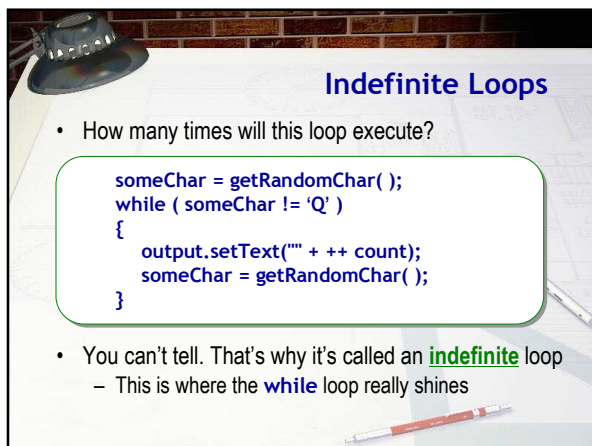
---

---

---

---

---



### Indefinite Loops

- How many times will this loop execute?

```
someChar = getRandomChar( );  
while ( someChar != 'Q' )  
{  
    output.setText("" + ++ count);  
    someChar = getRandomChar( );  
}
```
- You can't tell. That's why it's called an **indefinite** loop
  - This is where the **while** loop really shines

---

---

---

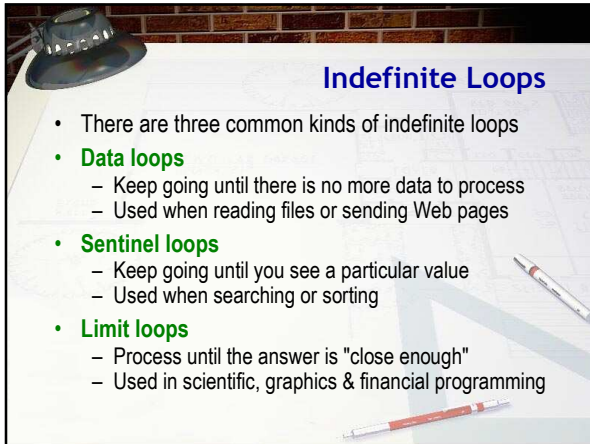
---

---

---

---

---



### Indefinite Loops

- There are three common kinds of indefinite loops
- **Data loops**
  - Keep going until there is no more data to process
  - Used when reading files or sending Web pages
- **Sentinel loops**
  - Keep going until you see a particular value
  - Used when searching or sorting
- **Limit loops**
  - Process until the answer is "close enough"
  - Used in scientific, graphics & financial programming

---

---

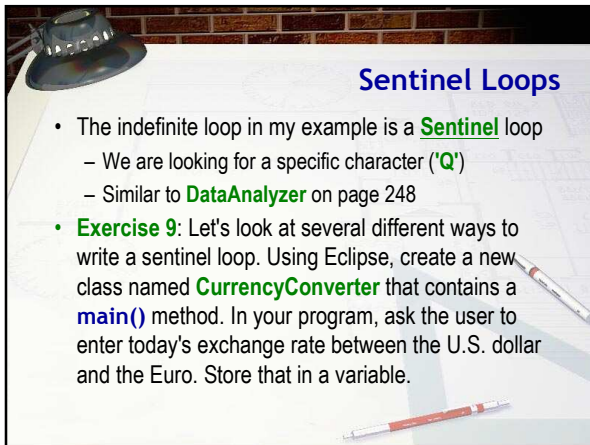
---

---

---

---

---



### Sentinel Loops

- The indefinite loop in my example is a **Sentinel** loop
  - We are looking for a specific character ('Q')
  - Similar to **DataAnalyzer** on page 248
- **Exercise 9:** Let's look at several different ways to write a sentinel loop. Using Eclipse, create a new class named **CurrencyConverter** that contains a **main()** method. In your program, ask the user to enter today's exchange rate between the U.S. dollar and the Euro. Store that in a variable.

---

---

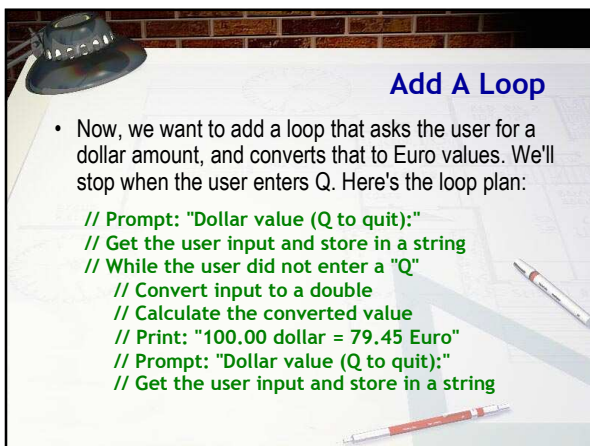
---

---

---

---

---



### Add A Loop

- Now, we want to add a loop that asks the user for a dollar amount, and converts that to Euro values. We'll stop when the user enters Q. Here's the loop plan:
  - // Prompt: "Dollar value (Q to quit):"
  - // Get the user input and store in a string
  - // While the user did not enter a "Q"
  - // Convert input to a double
  - // Calculate the converted value
  - // Print: "100.00 dollar = 79.45 Euro"
  - // Prompt: "Dollar value (Q to quit):"
  - // Get the user input and store in a string

---

---

---

---

---

---

---



### Primed Loops

- This kind of sentinel loop is called a **primed** loop
  - Initialize the test condition **before** the loop
    - Often this involves **reading** a value or getting input
    - This is called **priming** the loop
  - Test the value in the loop condition
  - Process the value in the body of the loop
  - Initialize the test condition at **bottom** of loop body
    - Preparing for the next repetition of the loop
    - Means "preparation" statements appear in two places

### Inline Tests

- One way to shorten this is using an **inline test**
    - Assign **and** test inside the loop condition
- ```
while (!(input = kbd.next()).equals("Q"))
{
    // loop statements here
}
```
- Only requires a single "read" (still need extra prompt)
  - Location of parentheses is critical
- Not that clear in this case

### Using a Boolean Flag

- Another way to write a sentinel loop is to use a Boolean flag in conjunction with an **if** statement:

```
boolean done = false;
while (! done)
{
    System.out.print("Dollar value (Q to quit): ");
    String input = kbd.next();

    if (input.equalsIgnoreCase("Q"))
    {
        done = true;
    }
    else
    {
        double dollar = Double.parseDouble(input);
        double euro = dollar * rate;
        System.out.printf("%.2f Dollar = %.2f Euro\n",
            dollar, euro);
    }
}
```

### Limit Loops

- Some loops look for a **limit** or **threshold**
  - The **waitForBalance()** method in Investment.java

```
//  
public void waitForBalance(double targetBalance)  
{  
    while (balance < targetBalance)  
    {  
        years++;  
        double interest = balance * rate / 100;  
        balance = balance + interest;  
    }  
}
```

---

---

---

---

---

---

---

---

### Intentional Bounds

- Suppose you search a **String** for the letter "F"
  - A sentinel bounds is the obvious choice

```
int i = 0;  
while (s.charAt(i) != 'F')  
    i++;  
// i contains the position of the 'F'
```

- This is called the **intentional** bounds

---

---

---

---

---

---

---

---

### Necessary Bounds

- What happens if the letter 'F' is not in the **String** s ?
  - Your program "crashes" when i becomes too big
- To fix this, you need to supply an **additional bounds** to use in case the value you are looking for is not found

```
int i = 0, len = s.length();  
while (i < len && s.charAt(i) != 'F') i++;  
// if i == len then 'F' not in String s
```

- This additional bounds is called a **necessary** bounds

---

---

---

---

---

---

---

---

### Test at the Bottom

- One more loop in addition to **for** and **while**
  - The **do-while** loop is an **exit-condition** loop
  - That means the test is located **after** the body

---

---

---

---

---

---

---

---

### When To Use Do-While?

- If you want to execute actions **before** making the test
  - Most of the time, you **don't** want to do that!

---

---

---

---

---

---

---

---

### Express Yourself

- Exercise 10:** Here is a limit loop that uses **do-while**. Rewrite it as a **while** loop.

```
int n = 1;
double x = 0;
double s;
do
{
    s = 1.0 / (n * n);
    x = x + s;
    n++;
} while (s > 0.01);
```

**Remember!**  
 n, s and x must  
 have the same  
 ending values in  
 both versions!

---

---

---

---

---

---

---

---

### Jump Statements

- Jumps are "low-level" flow of control statements
  - In contrast to "high-level" statements like if and loops
- A jump statement is sometimes called a **goto**
  - Although **goto** is a reserved word in Java, its not used
- Java does have two jump statements
  - **break** : used to "break" out of a loop (but not an if)
  - **continue** : used to "continue" or "start" a loop
- Java also has a third jump statement
  - Used to exit a **nested** loop




---

---

---

---

---

---

---

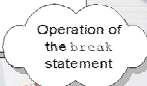
---

### The break Statement

- **break** jumps out of a **switch** or loop (but not an **if**)
- In a loop, **break** jumps to the first statement **following** the loop body as shown here.
- Can make your code clearer when used to construct a **loop-and-a-half**

```

while (n != 0)
{
    ...
    break;
    ...
}
statement;
statement;
  
```



Operation of the break statement

---

---

---

---

---

---

---

---

### A "Loop-and-a-half"

- A loop-and-a-half is available in languages like Ada
  - In Java, use **if** with **break** to do the same thing
  - Put the necessary bounds in your loop condition
  - Put the intentional bounds in the if statement
- **Exercise 11**: Open **BreakFor.java**, locate comments
  - Write a loop that visits every character in the **String**
  - Extract the current character
  - If the current character is a period, then **break**
  - Otherwise, append it to the answer
- Very useful for "primed" loops with complex setup

---

---

---

---

---

---

---

---

## Another Loop-and-a-Half

- Here's a sentinel loop replaced by loop-and-a-half

```
while (true)
{
    System.out.print("Dollar value (Q to quit): ");
    String input = kbd.next();

    if (input.equalsIgnoreCase("Q")) break;

    double dollar = Double.parseDouble(input);
    double euro = dollar * rate;
    System.out.printf("%.2f Dollar = %.2f Euro\n",
        dollar, euro);
}
```

## The continue Statement

- The **continue** statement only works inside loops
- Instead of leaving the loop, it starts the next iteration
- In **while** and **do-while** **continue** jumps to the loop test expression
- In **for**, it jumps to the update expression

```
a) while (n != 0)
{
    ...
    continue;
    ...
}

b) for (n = 0; n < 10; n++)
{
    ...
    continue;
    ...
}
```

Operation of  
the continue  
statement

## Completing ContinueFor

- Suppose you want to convert 1,234,456 to an **int**
  - Can't use **Integer.parseInt()** because of commas
- You **can** write a loop to convert the number manually
  - Use the **String**'s length as the loop bounds
  - Extract each character from the **String**
  - Use **Character.isDigit()** to see if it is a number
  - If not, then restart the loop using **continue**
  - If it is, then convert the digit into a binary number
  - Add it to the running total
- Exercise 12:** Complete, snap 2 pics (running/code)



### Nested Loops

- Loops can be **nested**, just like **if** statements
  - We call the nested loop the **inner** loop
  - We call the enclosing loop the **outer** loop

```

for (int outer = 1; outer <= 10; outer++) {
    for (int inner = 1; inner <= 10; inner++)
        System.out.printf("%4d", outer * inner);
    System.out.println();
}
```

- Exercise 13:** Open Dr. Java interaction pane. Type code above. Snap a screenshot.

---

---

---

---

---

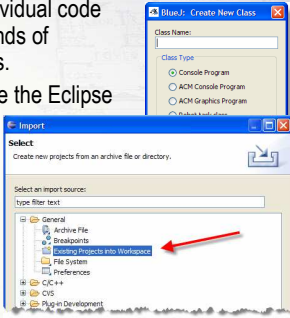
---

---

---

### Importing Projects

- Eclipse doesn't have individual code templates for different kinds of programs like BlueJ does.
- Let's see how we can use the Eclipse **Import Project** function to create a library of templates.
- Choose **File->Import** then **Existing Projects into Workspace**




---

---

---

---

---

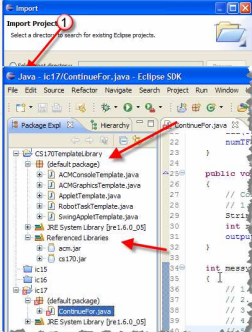
---

---

---

### Projects from Archives

- Click **Select Archive file**, then **Browse** button
- Locate and select the file: **CS170TemplateLibrary.zip** on the Q: drive then click the **Finish** button
- Exercise 14:** Expand default package and Referenced Libraries like this and snap a screen-shot.




---

---

---

---

---

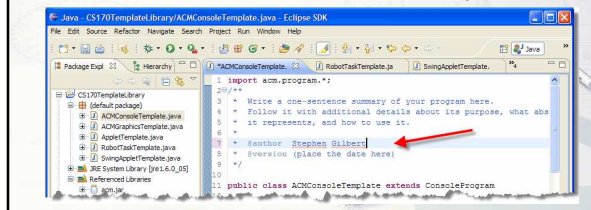
---

---

---

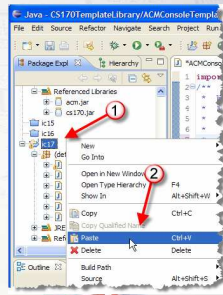
## Adding Comments

- When you use a template file, Eclipse won't be able to add the custom comments that you set up
  - No problem, you can just add them to the templates
- Exercise 15:** Open each template file, add your name



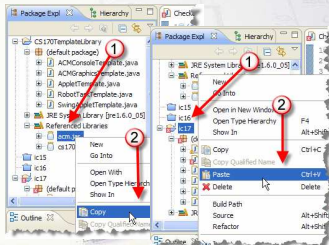
## Projects from Templates

- Let's create an ACM Graphics program with nested loops.
  - Select and then right-click **ACMGraphicsTemplate**, choose **Copy** from the menu.
  - Then, select the **ic13** project and choose **Paste**



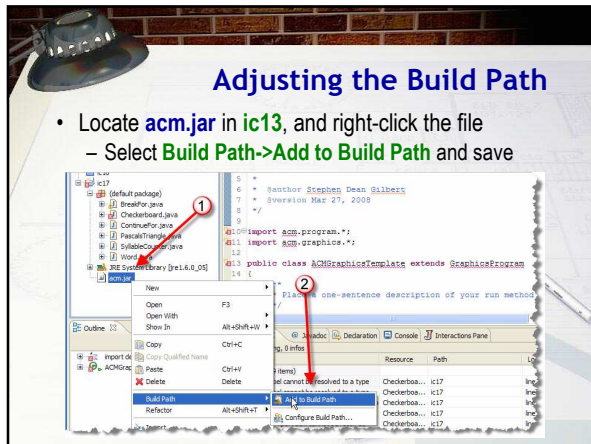
## Adding a Library File

- Code doesn't compile; there are **two** problems
  - Need the library file, **acm.jar** in the project
  - Need the class name to match the file name
- Right-click **acm.jar** in the Referenced libraries section of the template project and choose **Copy**
- Right-click **ic13** and click **Paste**



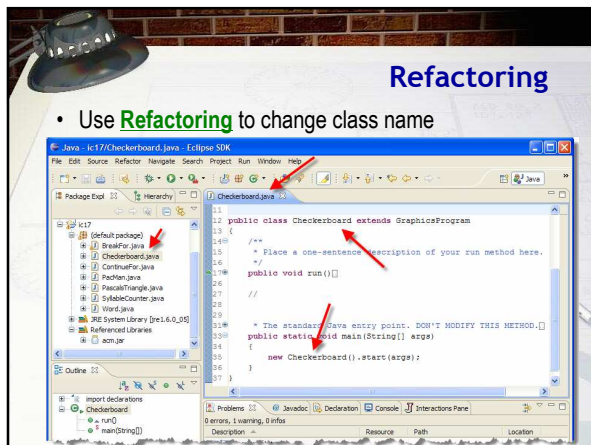
## Adjusting the Build Path

- Locate **acm.jar** in **ic13**, and right-click the file
  - Select **Build Path->Add to Build Path** and save



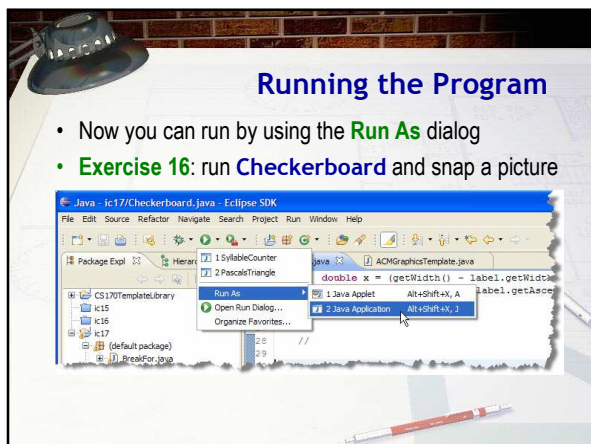
## Refactoring

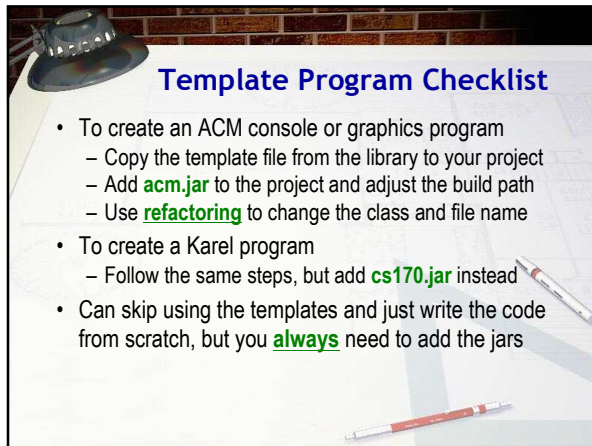
- Use **Refactoring** to change class name



## Running the Program

- Now you can run by using the **Run As** dialog
- Exercise 16:** run **Checkerboard** and snap a picture





### Template Program Checklist

- To create an ACM console or graphics program
  - Copy the template file from the library to your project
  - Add **acm.jar** to the project and adjust the build path
  - Use **refactoring** to change the class and file name
- To create a Karel program
  - Follow the same steps, but add **cs170.jar** instead
- Can skip using the templates and just write the code from scratch, but you **always** need to add the jars

---

---

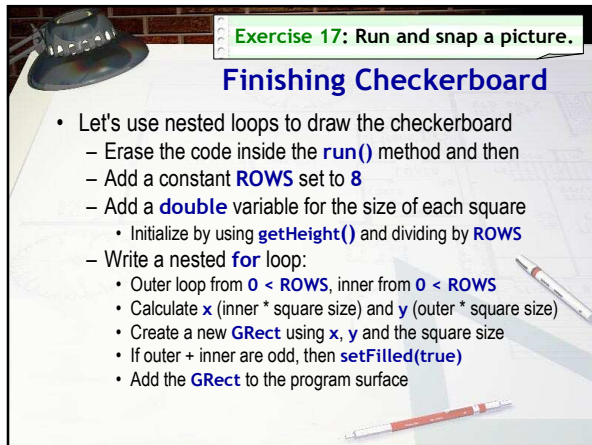
---

---

---

---

---



**Exercise 17: Run and snap a picture.**

### Finishing Checkerboard

- Let's use nested loops to draw the checkerboard
  - Erase the code inside the **run()** method and then
  - Add a constant **ROWS** set to **8**
  - Add a **double** variable for the size of each square
    - Initialize by using **getHeight()** and dividing by **ROWS**
  - Write a nested **for** loop:
    - Outer loop from **0 < ROWS**, inner from **0 < ROWS**
    - Calculate **x** (inner \* square size) and **y** (outer \* square size)
    - Create a new **GRect** using **x**, **y** and the square size
    - If outer + inner are odd, then **setFilled(true)**
    - Add the **GRect** to the program surface

---

---

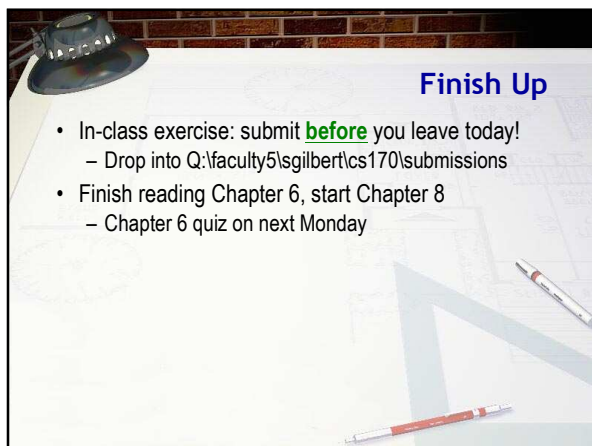
---

---

---

---

---



### Finish Up

- In-class exercise: submit **before** you leave today!
  - Drop into Q:\faculty5\sgilbert\cs170\submissions
- Finish reading Chapter 6, start Chapter 8
  - Chapter 6 quiz on next Monday

---

---

---

---

---

---

---